In [1]:
```
# Digital Image Processing - Spatial Filtering (CST 382-3 / IIT 361-2)
# Author: Your Name

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Helper function to display images
def show_images(images, titles, cols=2, cmap='gray'):
    rows = (len(images) + cols - 1) // cols
    plt.figure(figsize=(14, 4 * rows))
    for i in range(len(images)):
        plt.subplot(rows, cols, i + 1)
        plt.imshow(images[i], cmap=cmap)
        plt.title(titles[i], fontsize=12, fontweight='bold')
        plt.axis('off')
    plt.tight_layout()
    plt.show()

# Load grayscale image
img_path = 'Filter_STI/Filter_STI/Brain3.png'  # Replace with any image from your l
print(f"Processing image: {img_path}")
img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

# ------------------------
# TASK 1: Apply Individual Filters
# ------------------------

# 1. Averaging Filter (Box Filter)
kernel = np.ones((5, 5), np.float32) / 25
avg = cv2.filter2D(img, -1, kernel)

# 2. Gaussian Filter
gaussian = cv2.GaussianBlur(img, (5, 5), 0)

# 3. Median Filter
median = cv2.medianBlur(img, 5)

# 4. Laplacian Filter (Edge)
laplacian = cv2.Laplacian(img, cv2.CV_64F)
laplacian = cv2.convertScaleAbs(laplacian)

# 5. Sobel Filter (X and Y)
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=5)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=5)
sobelx = cv2.convertScaleAbs(sobelx)
sobely = cv2.convertScaleAbs(sobely)

# 6. Bilateral Filter
bilateral = cv2.bilateralFilter(img, 9, 75, 75)

# 7. Canny Edge Detection
canny = cv2.Canny(img, 100, 200)
```

```
# 8. Prewitt Filter
prewitt_kernelx = np.array([[1, 0, -1],
                            [1, 0, -1],
                            [1, 0, -1]])
prewitt_kernely = np.array([[1, 1, 1],
                            [0, 0, 0],
                            [-1, -1, -1]])
prewitt_x = cv2.filter2D(img, -1, prewitt_kernelx)
prewitt_y = cv2.filter2D(img, -1, prewitt_kernely)
prewitt = cv2.addWeighted(prewitt_x, 0.5, prewitt_y, 0.5, 0)

# 9. Scharr Operator
scharr_x = cv2.Scharr(img, cv2.CV_64F, 1, 0)
scharr_y = cv2.Scharr(img, cv2.CV_64F, 0, 1)
scharr = cv2.magnitude(scharr_x, scharr_y)
scharr = cv2.convertScaleAbs(scharr)

# 10. Roberts Cross Operator
roberts_cross_v = np.array([[1, 0],
                            [0, -1]])
roberts_cross_h = np.array([[0, 1],
                            [-1, 0]])
roberts_v = cv2.filter2D(img, -1, roberts_cross_v)
roberts_h = cv2.filter2D(img, -1, roberts_cross_h)
roberts = cv2.addWeighted(roberts_v, 0.5, roberts_h, 0.5, 0)

# Display all filter results
show_images(
    [img, avg, gaussian, median, laplacian, sobelx, sobely, bilateral, canny, prewi
    ['Original', 'Averaging Filter', 'Gaussian Filter', 'Median Filter', 'Laplacian
     'Sobel X', 'Sobel Y', 'Bilateral Filter', 'Canny Edge', 'Prewitt Filter', 'Sch
)

# ------------------------
# TASK 2: Compare Filters
# ------------------------

show_images(
    [img, gaussian, median, sobelx],
    ['Original', 'Gaussian', 'Median', 'Sobel X'],
    cols=2
)

print("\n🔍 Task 2 - Filter Comparison")
print("- Gaussian Filter: Smooths the image, reduces Gaussian noise, but may blur e
print("- Median Filter: Removes salt-and-pepper noise effectively while preserving
print("- Sobel Filter (X): Highlights vertical edges and fine structures.")

# ------------------------
# TASK 3: Combine Filters (Pipeline)
# ------------------------

# Pipeline 1: Median → Sobel X
median1 = cv2.medianBlur(img, 5)
sobel_from_median = cv2.Sobel(median1, cv2.CV_64F, 1, 0, ksize=5)
sobel_from_median = cv2.convertScaleAbs(sobel_from_median)
```

```
show_images(
    [img, median1, sobel_from_median],
    ['Original', 'After Median Filter', 'After Median + Sobel X']
)

print("\n🛠 Pipeline 1: Median + Sobel")
print("Reason: Median removes noise, and Sobel highlights edges without interferenc

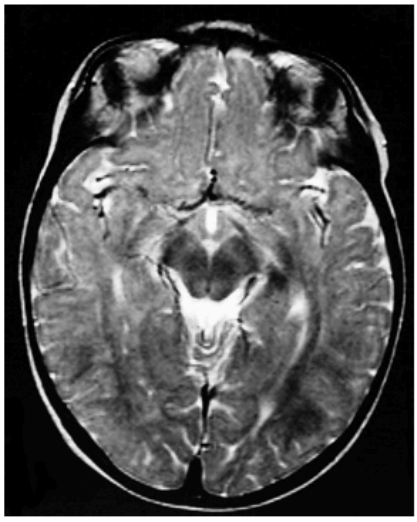
# Pipeline 2: Gaussian → Canny
gaussian2 = cv2.GaussianBlur(img, (5, 5), 0)
canny_after_gaussian = cv2.Canny(gaussian2, 100, 200)

show_images(
    [img, gaussian2, canny_after_gaussian],
    ['Original', 'After Gaussian', 'After Gaussian + Canny']
)

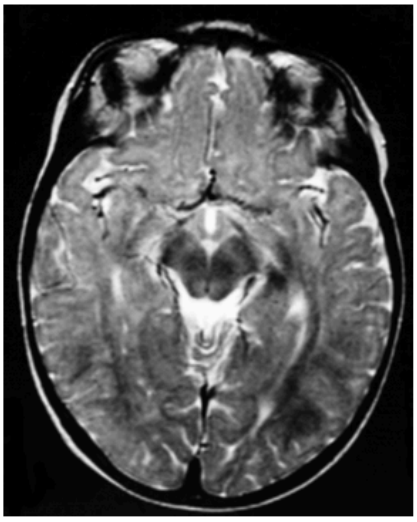
print("\n🛠 Pipeline 2: Gaussian + Canny")
print("Reason: Gaussian smooths noise, and Canny performs strong edge detection.")
```

```
Processing image: Filter_STI/Filter_STI/Brain3.png
```

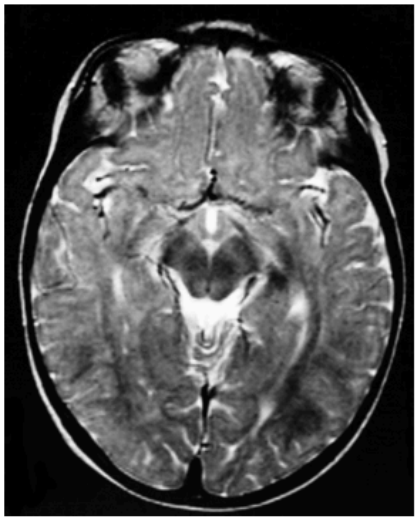
Original



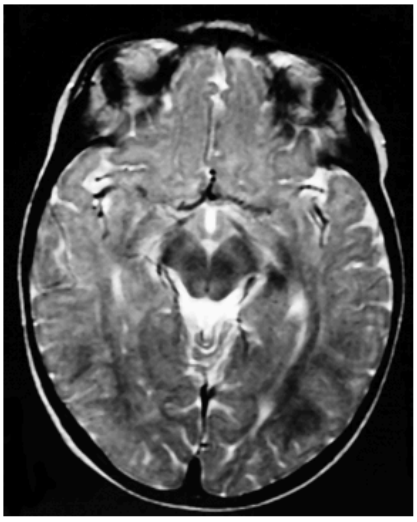
Averaging Filter



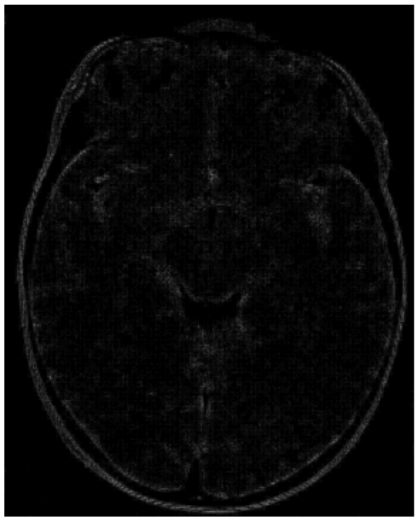
Gaussian Filter



Median Filter



Laplacian Filter



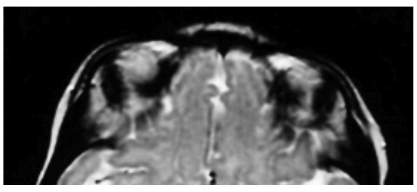
Sobel X



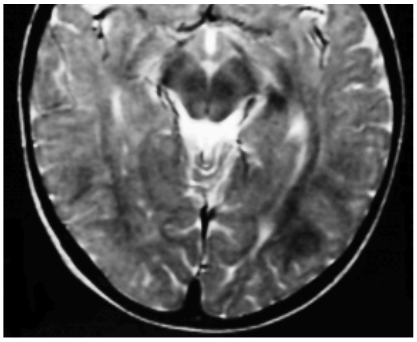
Sobel Y



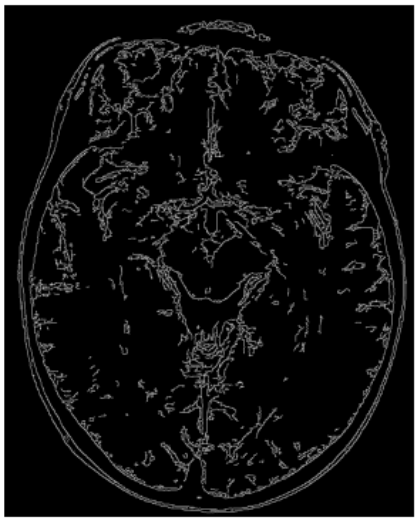
Bilateral Filter



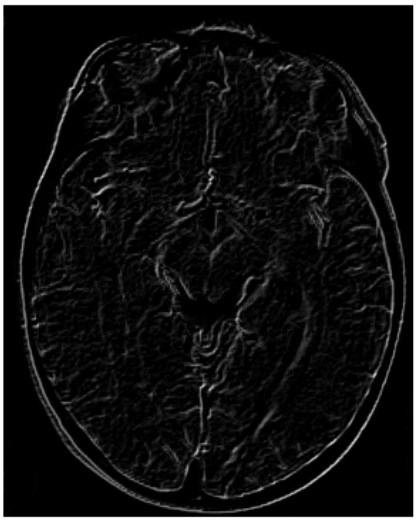




Canny Edge



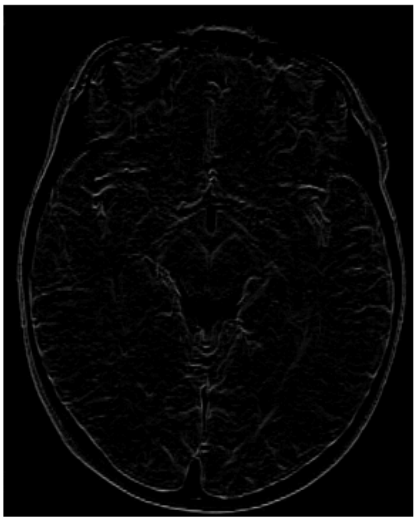
Prewitt Filter

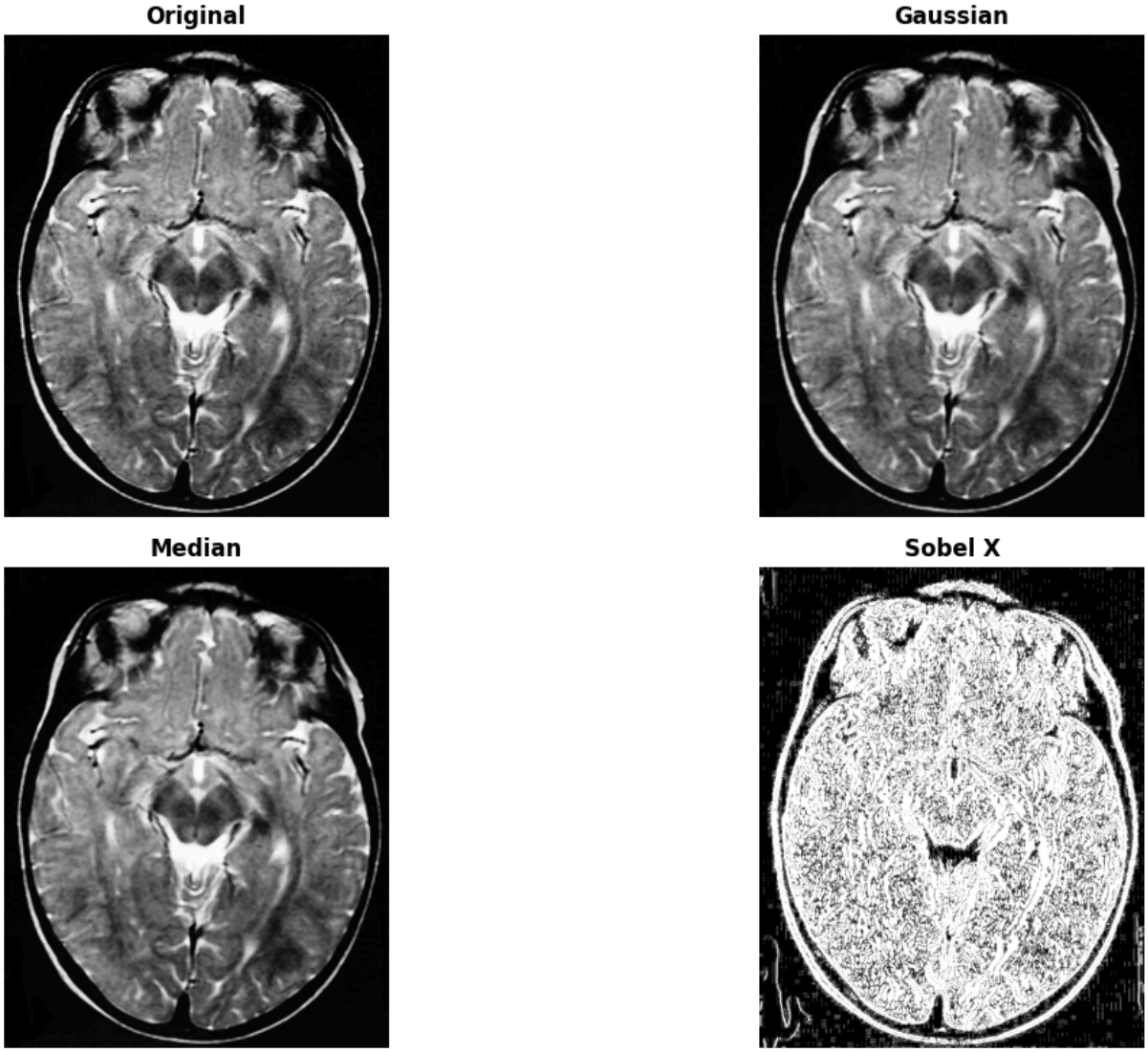


Scharr Filter



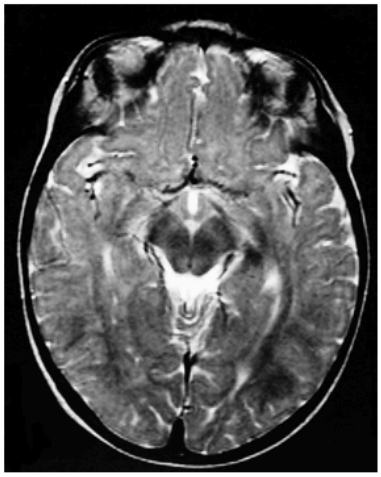
Roberts Cross



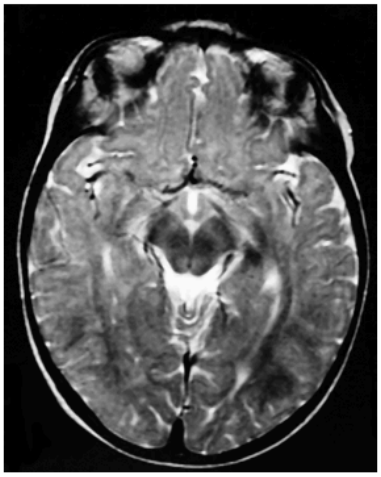


🔍 Task 2 - Filter Comparison
- Gaussian Filter: Smooths the image, reduces Gaussian noise, but may blur edges.
- Median Filter: Removes salt-and-pepper noise effectively while preserving edges.
- Sobel Filter (X): Highlights vertical edges and fine structures.

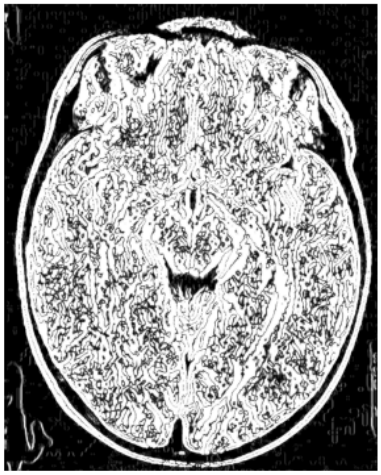
Original



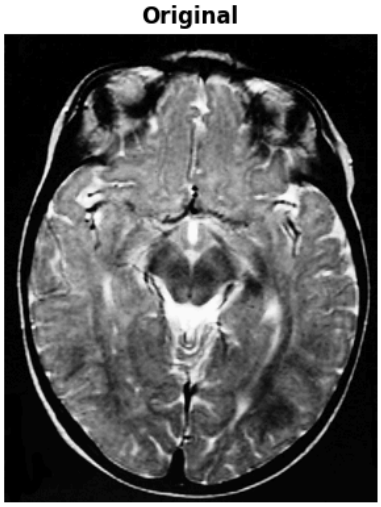
After Median Filter

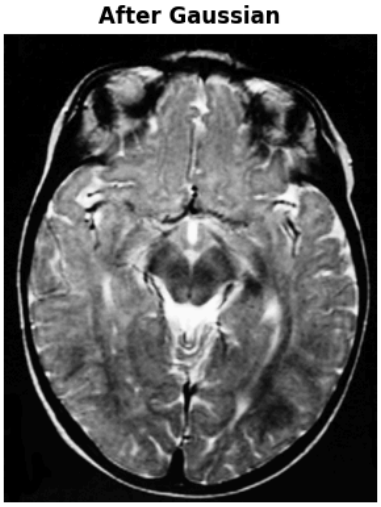


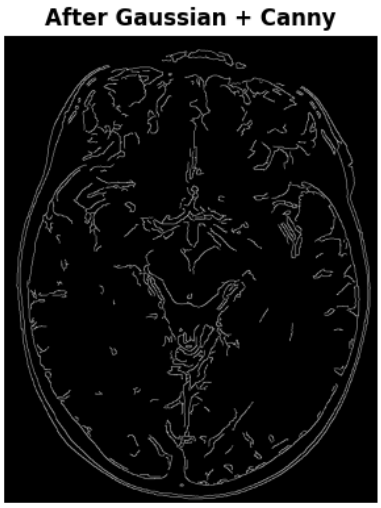
After Median + Sobel X



🛠 Pipeline 1: Median + Sobel
Reason: Median removes noise, and Sobel highlights edges without interference.







🛠 Pipeline 2: Gaussian + Canny
Reason: Gaussian smooths noise, and Canny performs strong edge detection.

In [ ]: